ENTERPRISE AI

Supply Chain Control Room

Detailed Project Specification, Architecture & Implementation Guide

Multi-Agent Order Fulfilment • Exception Handling • Knowledge Graph • Auditability

Version 1.0 | 14 October 2026

Implementation-status warning: This document describes the target design and identifies areas previously reported as prototype, simulated, starter, or not fully connected. Verify all status claims against the current repository before external submission.

Document Map

- 1. Executive Summary
- 2. Problem Statement and Background
- 3. Objectives, Scope, and Guardrails
- 4. Definition of Done and Acceptance Criteria
- 5. Roles and Responsibilities
- 6. System Architecture and Data Flow
- 7. Frontend Structure and Implementation Steps
- 8. Backend Service Design and API Contracts
- 9. Multi-Agent Design: Five Agents
- 10. Workflow States and Exception Handling
- 11. Demonstration: ORD-1042 / WP-800
- 12. Data Model and Persistence
- 13. Knowledge Graph and Retrieval
- 14. Auditability and Blockchain
- 15. API and Realtime Design
- 16. Security, Reliability, and Testing
- 17. Local Setup and Configuration
- 18. Deployment, Observability, and Benchmarks
- 19. Implementation Roadmap
- 20. Final Demo Checklist and Documentation Artifacts
- Appendix A. Status Legend

1. Executive Summary

Enterprise AI Supply Chain Control Room is a multi-agent workflow system for coordinating order fulfilment across inventory, procurement, finance, and logistics. A Supervisor agent coordinates specialist agents, surfaces operational exceptions, and presents the workflow in a shared control-room dashboard.

The system is intended to make operational decisions traceable: an operator should be able to inspect the order, specialist findings, exception causes, final workflow state, and audit events that explain what happened.

Status caveat: prior project context indicates that the React/TypeScript frontend prototype is the most complete layer. The backend, agent orchestration, database, knowledge graph, blockchain, and live event integrations have previously been described as starter, simulated, planned, or not fully connected. This document separates target design from verified implementation; confirm the current repository before claiming a feature is production-ready.

2. Problem Statement and Background

Order fulfilment crosses multiple functions. Inventory may not cover the requested quantity; procurement may find no eligible supplier; the proposed purchase may exceed budget; logistics may predict a late delivery. When each function operates in isolation, teams discover conflicts late, manually transfer information, and struggle to explain how a final decision was reached.

Operational problems addressed

- Fragmented order, stock, supplier, finance, and shipment state.
- Late detection of supplier unavailability, inventory shortages, budget breaches, and delivery delays.
- Manual hand-offs without a single trace of the decision path.
- Limited visibility into agent actions, exceptions, and the evidence used.
- Difficulty distinguishing a successful workflow from a simulated or partially completed workflow.

Project objective

Provide one operational interface and workflow engine that coordinates specialist checks, detects exceptions, records decisions, and gives users a clear explanation of the current state and next action.

3. Objectives, Scope, and Guardrails

Objectives

- Coordinate order fulfilment through a Supervisor and four specialist agents.
- Represent inventory, procurement, finance, and logistics as distinct capabilities.
- Detect supplier unavailable, budget breach, delivery delay, and compound exceptions.
- Expose orders, agent activity, workflow status, exceptions, knowledge-graph relationships, and audit history in a dashboard.
- Persist operational data and workflow evidence through a structured backend.
- Keep calculations and approval rules deterministic, even when LLMs help interpret or summarise information.

In scope

- Order workflow simulation and scenario-based testing.
- Inventory checks and replenishment recommendations.
- Supplier evaluation and procurement proposals.
- Budget validation and finance approval signals.
- Shipment planning and delivery-delay alerts.
- Agent orchestration, exception tracking, audit views, graph view, and operational benchmarks.
- Replacing demo state with real API-backed data incrementally.

Guardrails / not assumed to be implemented

- No unattended real-world purchasing or payment execution unless separately authorised and built.
- No claim of live ERP, WMS, TMS, or supplier portal integration without working adapters.
- No claim of blockchain verification merely because the UI displays a blockchain-style screen.
- No claim that Kubernetes is deployed unless manifests and a working environment are verified.
- No LLM-authored unrestricted SQL or unreviewed high-impact tool calls.

4. Definition of Done and Acceptance Criteria

#	Acceptance criterion	How to verify
1	Unified dashboard	Navigate Orders, Inventory, Procurement, Finance, Logistics, Exceptions, Knowledge Graph, Audit Trail, and Benchmarks.
2	End-to-end order run	Run sample ORD-1042 and inspect its workflow state and final outcome.
3	Separate specialist agents	Show different structured outputs from Inventory, Procurement, Finance, and Logistics; Supervisor combines them.
4	Exception scenarios	Trigger supplier unavailable, budget breach, delivery delay, and compound exception; inspect cause and recommended next step.
5	Persistent state	Restart backend and verify saved orders, workflow runs, exceptions, and audit events remain available once persistence is implemented.
6	Audit trace	Inspect event type, timestamp, actor/agent, entity ID, workflow run ID, and outcome.
7	Knowledge graph	Inspect order/product/supplier/warehouse/shipment relationships and trace nodes to source records.
8	Safe failures	Simulate invalid input, dependency outage, timeout, and malformed agent output; workflow must fail safely.
9	Security	Verify server-side access checks and ensure secrets are not committed.
10	Reproducibility	A new developer follows documented setup, starts services, and runs tests.

The frontend has previously been described as an interactive prototype with 11 screens, shared state, scenario simulation, activity/audit logs, and knowledge-graph/blockchain-style UI. A production build was previously reported to pass. Re-run the build and verify each integration before using those as current claims.

5. Roles and Responsibilities

Role / component	Responsibility	Boundary
Operations user	Monitor orders, inspect recommendations, manage permitted actions, resolve assigned exceptions.	Cannot bypass server-side policy or rewrite audit history.
Supervisor agent	Coordinate specialist agents, aggregate results, select next state, escalate unresolved cases.	Must rely on structured evidence; should stop or escalate when outputs conflict.
Inventory agent	Check on-hand, reserved, available stock, warehouse options, and replenishment need.	Does not approve spend or claim physical receipt without evidence.
Procurement agent	Compare eligible suppliers, price, availability, and lead time; prepare a proposal.	A proposal is not a purchase approval or completed purchase.
Finance agent	Calculate projected spend and apply budget/approval policy.	Budget arithmetic and approval checks must be deterministic.
Logistics agent	Evaluate shipment options, delivery plan, ETA, and delay risk.	Simulated ETA must not be labelled live carrier tracking.
Auditor / reviewer	Inspect decisions, evidence, workflow trace, and audit records.	Read-only access to the audit trail.

These are target operating roles. If the current application has only a demo user or lacks authentication, describe RBAC as planned rather than implemented.

6. System Architecture and Data Flow

Target stack by layer

Layer	Technology	Purpose
Frontend	React, TypeScript, Tailwind CSS	Control-room screens and typed client state.
API	Python, FastAPI	Validated endpoints, authorization, domain-service calls.
Orchestration	LangGraph	Explicit workflow state, agent routing, conditional branches, checkpoints.
LLM provider	Configured provider such as Gemini, if selected	Interpretation and synthesis; not the source of truth for facts or calculations.
Relational database	PostgreSQL	Orders, inventory, proposals, shipments, workflow runs, exceptions, audit metadata.
Vector retrieval	pgvector, if enabled	Similarity search over approved policy or operational documents.
Knowledge graph	Neo4j / Cypher	Relationships among orders, products, suppliers, warehouses, and shipments.
Audit anchor	Solidity / EVM network / Web3.py, if enabled	Optional hash anchoring for tamper evidence.
Realtime	WebSockets, if enabled	Push workflow and exception changes to connected clients.
Runtime	Docker; Kubernetes as an option	Reproducible local services and potential deployment orchestration.

Request path

1. Operator selects an order or starts a scenario in the React UI.
2. Frontend service boundary calls the API; prior context names apiServices.ts and DemoContext.tsx in the prototype.
3. Backend validates input and creates a workflow run ID and correlation ID.

4. Supervisor reads authoritative state and routes work to specialist agents.
5. Specialists return structured results, evidence references, warnings, and status codes.
6. Supervisor evaluates results and selects continue, exception, human review, or safe stop.
7. Backend persists state and appends audit events; graph/vector/blockchain adapters run only when configured.
8. API response updates the UI; WebSocket events can push subsequent updates if that integration is live.

Keep presentation, HTTP routes, business rules, agent orchestration, and persistence adapters separate. This prevents business logic from being duplicated across UI components and makes simulated services replaceable.

7. Frontend Structure and Implementation Steps

Known screens from prior context include Orders, Inventory, Procurement, Finance, Logistics, Knowledge Graph, Audit Trail, Exceptions, and Benchmarks, plus scenario simulation. The prototype was also described as having 11 screens; confirm the exact route list in the current branch.

Known file / module	Expected responsibility
apiServices.ts	Central boundary for API calls; isolate components from transport details.
DemoContext.tsx	Shared demo/workflow state; must not compete with backend as source of truth after integration.
Orders	Order list, details, filters, and workflow launch.
Inventory	Stock, allocation, availability, and replenishment state.
Procurement	Supplier options, quotes, sourcing decisions, and proposals.
Finance	Budget calculation inputs, decision, and approval state.
Logistics	Shipment plan, ETA, delivery state, and delay risk.
Knowledge Graph	Relationship view with source entity links.
Audit Trail	Chronological trace filtered by order/workflow run.
Exceptions	Cause, severity, owner, status, evidence, and resolution.
Benchmarks	Operational metrics with documented definitions.

Step-by-step frontend work

1. Inventory all routes, components, context providers, mock fixtures, and API calls before changing code.
2. Define shared TypeScript types for Order, InventoryItem, Supplier, PurchaseProposal, BudgetDecision, Shipment, WorkflowRun, AgentRun, ExceptionRecord, and AuditEvent.
3. Identify which pages read from DemoContext.tsx and which call apiServices.ts.
4. Keep demo fixtures behind a clearly labelled demo adapter; do not silently fall back to mocks when a live API fails.
5. Add loading, empty, success, partial-result, and error states to every API-backed screen.
6. Use stable IDs and workflow correlation IDs to link order details, exceptions, agent events, and audit entries.
7. Ensure filters, counts, and detail pages use the same source and definitions.
8. Run the declared lint, type-check, and production-build scripts after each integration change.

8. Backend Service Design and API Contracts

Service boundaries

• Order service: order reads, permitted updates, and transition validation.

• Inventory service: availability, reservation, allocation, replenishment calculations.

• Procurement service: supplier eligibility, quote comparison, sourcing proposals.

• Finance service: budget lookup, deterministic spend calculation, approval decision.

• Logistics service: shipment options, ETA/risk calculation, delivery-state updates.

• Workflow service: start/resume LangGraph runs, checkpoint state, correlate results.

• Exception service: create, assign, acknowledge, resolve, deduplicate, and reopen.

• Audit service: append events and expose read-only timelines.

• Adapters: PostgreSQL, Neo4j, vector retrieval, model provider, WebSocket bus, optional blockchain.

Request handling sequence

1. Validate schema and authenticated identity.
2. Check authorization and record scope before reading or changing protected data.
3. Create or propagate request/workflow correlation IDs.
4. Call a domain service; keep business rules out of route handlers.
5. Use database transactions when operational state and audit events must agree.
6. Call external services through adapters with timeouts, bounded retries, and explicit failure handling.
7. Return typed outcomes, reason codes, current state, and correlation ID.
8. Log structured diagnostics without secrets or unnecessary sensitive payloads.

Illustrative route	Purpose
GET /api/health	Service health/readiness.
GET /api/orders	Filtered, paginated order list.

Illustrative route	Purpose
GET /api/orders/{order_id}	Order details and latest workflow summary.
POST /api/orders/{order_id}/runs	Start a workflow run.
GET /api/workflows/{run_id}	Workflow state and timestamps.
GET /api/workflows/{run_id}/trace	Ordered agent outputs and events.
GET /api/exceptions	Filter exception register.
POST /api/exceptions/{id}/acknowledge	Acknowledge an exception.
POST /api/exceptions/{id}/resolve	Resolve with reason/evidence.
GET /api/audit?entity_id=...	Read an entity's audit timeline.
GET /api/graph/orders/{order_id}	Fetch graph neighbourhood.

These paths are proposed contracts, not a claim that the exact endpoints already exist. Align documentation with the actual FastAPI routes.

9. Multi-Agent Design: Five Agents

Supervisor agent

1. Accept the workflow request and create a run ID.
2. Load order state, policy configuration, and available evidence.
3. Choose the required specialist checks; run independent checks in parallel only when dependencies permit.
4. Validate each result and identify missing, conflicting, or failed outputs.
5. Select continue, request correction, open exception, request human review, or stop safely.
6. Summarise the outcome using evidence returned by specialists.
7. Append a decision event and preserve the reasoning inputs/references.

Inventory agent

1. Read product, requested quantity, on-hand quantity, reservations, available quantity, and warehouse state.
2. Calculate fulfilment gap in deterministic application code.
3. Return quantities, calculation basis, availability status, and feasible options.
4. Create an inventory exception if policy conditions are not met.
5. Never treat an LLM recommendation as proof that stock was physically received.

Procurement agent

1. Receive the inventory gap and sourcing constraints.
2. Compare eligible suppliers using price, availability, lead time, and configured preference/risk data.
3. Return supplier options, estimated cost, currency, lead time, assumptions, and source references.
4. If no eligible supplier exists, return Supplier Unavailable rather than inventing a quote.
5. Send the proposal to Finance before any purchase is considered approved.

Finance agent

1. Load the applicable budget, committed spend, proposal cost, currency, and approval thresholds.
2. Calculate projected spend using deterministic code and documented rounding rules.
3. Return within-budget, budget-breach, or human-approval-required with the exact calculation inputs.
4. Do not approve because an LLM says the spend appears reasonable.
5. Record budget version, calculation values, and decision evidence.

Logistics agent

1. Read fulfilment source, destination, shipping options, capacity, and delivery constraints.
2. Evaluate the delivery plan from structured data or configured estimates.
3. Raise Delivery Delay if predicted/reported arrival violates the order window.
4. Return plan, ETA, risk, assumptions, and operator action.
5. Do not describe a simulated ETA as carrier-connected tracking.

Common agent output contract

Each result should contain agent_name, workflow_run_id, entity_id, status, summary, structured_findings, evidence_refs, warnings, started_at, completed_at, and error details if applicable. Use enumerated statuses and reason codes, not free-text alone.

10. Workflow States and Exception Handling

Suggested workflow lifecycle

1. RECEIVED — request accepted.
2. VALIDATING — schema, order status, and policy inputs checked.
3. INVENTORY_CHECK — availability evaluated.
4. PROCUREMENT_CHECK — sourcing evaluated when needed.
5. FINANCE_CHECK — projected spend checked.
6. LOGISTICS_PLAN — delivery feasibility and risk evaluated.
7. READY_FOR_FULFILMENT — required checks pass and approvals are complete.
8. EXCEPTION_OPEN — a blocking/review condition exists.
9. AWAITING_HUMAN_REVIEW — policy requires an operator decision.
10. COMPLETED — configured objective reached.
11. FAILED_SAFE — technical or policy failure stopped progression.

Treat these names as a recommended state model. Match the actual enums and transition table to the code.

Exception record fields

- exception_id, workflow_run_id, order_id, and relevant entity IDs.
- type/reason_code, severity, summary, supporting facts, evidence references.
- detected_by, detected_at, updated_at, owner/team, acknowledgement status.
- resolution status, resolution notes, resolving actor, and resolution time.
- human-approval requirement and approver reference, where applicable.
- deduplication key so retries do not create duplicate open exceptions.

Scenario	Expected behaviour
Supplier Unavailable	No eligible supplier: record the cause, do not fabricate a quote, request alternative sourcing or human action.
Budget Breach	Projected spend exceeds configured limit: block automatic approval and expose calculation and threshold.
Delivery Delay	Delivery estimate violates the order window: raise exception and evaluate permitted alternatives.
Compound Exception	Preserve every cause, prioritise, avoid contradictory actions, and let Supervisor coordinate.
Agent/service failure	Bounded retry if appropriate; record failure; missing output must never count as approval.

11. Demonstration: ORD-1042 / WP-800

Prior project context identifies ORD-1042 and WP-800 as deterministic sample references. Verify the exact meaning of WP-800 in the current fixture data before describing it as a supplier, product, or other entity.

1. Open Orders and select ORD-1042; confirm its fields come from the configured scenario data source.
2. Start the workflow and capture workflow_run_id/correlation_id.
3. Supervisor validates the order and initiates specialist checks.
4. Inventory returns available quantity, requested quantity, and any gap.
5. If a gap exists, Procurement returns a sourcing proposal or Supplier Unavailable exception.
6. Finance checks proposal cost against the applicable budget and records the calculation.
7. Logistics returns delivery feasibility and flags a delay if the expected window is violated.
8. Supervisor combines the results and advances only when required checks pass; otherwise it opens or updates exceptions.
9. Open Exceptions and verify each issue has type, severity, owner, reason, and evidence.
10. Open Audit Trail and trace events by order ID, workflow run ID, timestamp, and actor.
11. Open Knowledge Graph and inspect relationships tied to the same scenario.
12. Repeat with supplier, budget, delivery, and compound exception fixtures and confirm the output changes for the intended reason.

The demo should explicitly state which results are simulated and which are generated by live services. Demonstrate a negative path as well as a successful path.

12. Data Model and Persistence

PostgreSQL should remain the source of truth for transactional state. Neo4j and pgvector supplement it; they should not become competing authorities.

Suggested entity/table	Important fields / purpose
orders	order_id, product, quantity, destination, requested date, status.
products	product_id, SKU, name, product attributes.
inventory_balances	product_id, warehouse_id, on_hand, reserved, available, updated_at.
warehouses	warehouse_id, location, capabilities.
suppliers	supplier_id, eligibility, active status, lead-time metadata.
supplier_quotes	quote_id, supplier_id, product_id, quantity, price, currency, lead time, validity.
purchase_proposals	proposal_id, order_id, supplier_id, total estimate, status, finance decision.
budgets	scope, amount, currency, effective dates, approval thresholds.
shipments	shipment_id, order_id, origin, destination, tracking reference, ETA, status.
workflow_runs	workflow_run_id, order_id, graph/version, status, timestamps.
agent_runs	agent_run_id, workflow_run_id, agent, input reference, structured output, status, timing.
exceptions	exception_id, workflow_run_id, type, severity, owner, status, dedupe key.
audit_events	event_id, entity, event type, actor, timestamp, payload/hash, workflow reference.
knowledge_chunks	source ID, version, content, access metadata, embedding; only if vector retrieval is enabled.
blockchain_anchors	event batch hash, transaction hash, network, block reference, confirmation state; if enabled.

Persistence rules

- Use primary/foreign keys, unique constraints, and indexes for frequent filters.
- Use transactions for state changes that must agree with audit events.
- Use migrations, not ad hoc production schema edits.
- Store timestamps consistently in UTC and render locally in the UI.
- Record model/provider/version and prompt/template version when AI output affects a decision.
- Keep evidence references so a reviewer can trace a decision to the data used.
- Never commit credentials in seed data or source control.

13. Knowledge Graph and Retrieval

Neo4j graph design

Potential node labels: Order, Product, Supplier, Warehouse, PurchaseProposal, Shipment, Exception, and AgentRun. Potential relationships: CONTAINS_PRODUCT, SOURCED_FROM, STORED_AT, FULFILLED_BY, SHIPPED_AS, TRIGGERED, and RECOMMENDED_BY. Use stable relational IDs as graph properties.

1. Define labels, relationship types, uniqueness constraints, and stable IDs.
2. Write parameterised Cypher; do not interpolate untrusted text into query strings.
3. Seed a deterministic graph that mirrors ORD-1042.
4. Build idempotent sync from PostgreSQL records to Neo4j.
5. Let users navigate from a graph node to its source record.
6. Add checks for missing nodes and stale relationships.

Vector retrieval, if enabled

- Choose approved policy or operational documents and preserve source IDs/versions.
- Chunk documents and attach access metadata.
- Generate embeddings and store them in pgvector.
- Apply access filters before sending retrieved text to a model.
- Return citations/source IDs with retrieved content.
- Use retrieval to explain policies; use deterministic code for quantities, budgets, permissions, and state transitions.
- Never let an LLM generate and execute unrestricted SQL.

14. Auditability and Blockchain

The database audit log should remain useful if blockchain is unavailable. Blockchain can optionally anchor event hashes to make later tampering detectable; it does not prove that the original business event was true.

Minimum audit event

- event_id, event_type, entity_type, entity_id.
- workflow_run_id/correlation_id and actor type/ID.
- timestamp, schema version, outcome, and reason code.
- Input/evidence references and canonical payload hash.
- Optional previous-event or batch reference.

Target anchoring workflow

1. Canonicalise selected events with stable field ordering and encoding.
2. Compute a cryptographic hash per event or batch.
3. Submit the hash and minimal non-sensitive metadata to the configured EVM-compatible network.
4. Store transaction hash, network, block reference, and confirmation state in PostgreSQL.
5. Recompute the hash during verification and compare it with the anchored value.
6. Handle pending/reverted/unavailable transactions without silently reporting success.

Do not store confidential business data on a public chain. Label audit events as unanchored, pending, confirmed, or verification-failed based on actual adapter results.

15. API and Realtime Design

Illustrative WebSocket event types: workflow.started, workflow.state_changed, agent.started, agent.completed, agent.failed, exception.created, exception.updated, and audit.event_appended.

- Each event should include event ID, emitted timestamp, workflow run ID, entity ID, and schema version.
- Clients must tolerate reconnects and missed events.
- After reconnect, refresh authoritative state from the API rather than assuming every WebSocket message arrived.
- Use idempotent event IDs to avoid duplicate UI activity entries.
- Keep realtime optional: core workflows should still be inspectable through normal API reads.

API error contract

Return a consistent error object with code, human-readable message, correlation_id, and optional field_errors. Do not expose stack traces, credentials, database details, or raw provider exceptions to end users.

16. Security, Reliability, and Testing

Security and reliability controls

- Validate request bodies and query parameters with typed schemas.
- Authenticate users and enforce authorization on the server; hiding a button is not authorization.
- Keep API keys, database passwords, JWT secrets, RPC URLs, and signing keys in environment variables or a secret manager.
- Use least-privilege database and blockchain identities.
- Use timeouts and bounded retries; retry operations must be idempotent.
- Validate agent outputs before they can change workflow state.
- Use deterministic code for arithmetic, stock quantities, permissions, and approvals.
- Treat retrieved text and user input as untrusted; defend against prompt injection.
- Do not let models choose arbitrary tools, unrestricted SQL, or arbitrary blockchain transactions.
- Redact secrets from logs and define safe failure states.

Test level	Test examples	Pass condition
Unit	Budget boundaries, stock availability, transition rules, exception classification.	Deterministic output for normal and boundary inputs.
Agent contract	Malformed JSON, missing evidence, timeout, contradictory findings.	Invalid outputs rejected or safely handled.
Workflow	ORD-1042, supplier unavailable, budget breach, delay, compound case.	Expected nodes, exceptions, and final state.
API	Authorization, validation, not-found, conflict, pagination.	Correct responses and no cross-scope data leakage.
Database	Migrations, constraints, transactions, idempotency.	No orphaned records or duplicate state.
Integration	FastAPI/PostgreSQL, Neo4j, model provider, WebSocket, chain adapter.	Works end-to-end or degrades safely.
UI	Loading/error/empty states, scenario toggles, audit drill-down.	UI matches backend state and labels simulation.
Release	Clean install, type check, lint, build, tests.	Documented commands pass on a clean checkout.

Critical negative tests

- Finance rejects but Supervisor attempts to mark the order ready.
- Supplier information is missing or stale.
- Agent returns prose instead of required structured output.
- The same workflow-start request is submitted twice.
- Neo4j, model provider, WebSocket, or RPC service is unavailable.
- Exception is resolved without required reason/evidence.
- Prompt injection attempts to override policy or access another scope.

17. Local Setup and Configuration

The exact repository scripts and folder layout have not been inspected in this turn. Treat these as templates and follow the current package manifests, lockfiles, README, and Compose file as the source of truth.

Setup sequence

1. Clone the repository and enter its root directory.
2. Read README; inspect frontend/backend folders, dependency files, environment examples, migrations, and Compose configuration.
3. Use the Node.js and Python versions required by the project.
4. Create a local environment file from the documented example and configure only services needed for the selected mode.
5. Start required dependencies and apply database migrations.
6. Load documented development fixtures; do not mix demo fixtures with production records.
7. Check health endpoints and connectivity before starting a workflow.
8. Run one happy-path workflow and one negative scenario.

Frontend command template

```
npm install
npm run dev
npm run build
```

Use the package manager and scripts declared in package.json. Do not mix npm, pnpm, yarn, or bun lockfiles.

Backend command template

```
python -m venv .venv
# Windows PowerShell: .venv\\Scripts\\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

The import path app.main:app is illustrative; replace it with the actual FastAPI module.

Config category	Example placeholder	Control
Frontend API	VITE_API_BASE_URL=http://localhost:8000	Public config only; never put secrets in VITE_* variables.
Database	DATABASE_URL=postgresql://...	Separate credentials per environment.
LLM provider	LLM_PROVIDER / LLM_API_KEY / LLM_MODEL	Make provider/model explicit in run metadata.
Neo4j	NEO4J_URI / NEO4J_USERNAME / NEO4J_PASSWORD	Optional if graph adapter is enabled.
WebSocket	WS_BASE_URL	Optional; client must recover from disconnect.
Blockchain	CHAIN_ID / RPC_URL / CONTRACT_ADDRESS	Optional; protect signing keys.
Security	JWT_SECRET / CORS_ORIGINS	Use strong secrets and restrictive origins.

18. Deployment, Observability, and Benchmarks

Deployment sequence

1. Freeze interface contracts and mark live versus simulated adapters.
2. Build and test frontend/backend independently.
3. Apply repeatable migrations and development seed scripts.
4. Containerise services that require consistent runtime dependencies.
5. Configure secrets outside the repository.
6. Run unit, workflow, API, integration, and security checks in CI.
7. Deploy to staging with non-sensitive data.
8. Verify health, logs, traces, exception resolution, and recovery from outages.
9. Gate real external actions behind explicit permissions and human approval.
10. Release with rollback instructions and version records for code, schema, workflow, and prompts.

Useful operational metrics

- Workflow completion and safe-failure rates.
- Median and p95 workflow duration.
- Agent latency, retry count, timeout count, structured-output failures.
- Open exceptions by type, severity, owner, and age.
- Supplier-unavailability frequency and fallback outcome.
- Budget-breach frequency and amount above threshold.
- Delivery-delay rate and ETA error when actual timestamps exist.
- Audit-write failures, graph-sync backlog, and blockchain-anchor status if enabled.

Every benchmark needs a definition, numerator/denominator, time window, source, and refresh cadence. Do not label a metric real-time unless its data path supports that claim. Kubernetes remains a deployment option unless actual deployment is verified.

19. Implementation Roadmap

Phase	Work	Exit criteria
1. Baseline	Inspect repo, scripts, screens, mock data, and current status.	Clean reproducible build and current-vs-planned feature matrix.
2. Contracts	Define shared TypeScript/Pydantic models, IDs, enums, API errors.	Consistent frontend/backend contracts.
3. Persistence	PostgreSQL schema, migrations, repositories, fixtures.	Orders, runs, exceptions, audit records persist.
4. API integration	Connect UI services to FastAPI; add auth and errors.	Dashboard reads authoritative API data.
5. Orchestration	Implement LangGraph nodes, transitions, checkpoints.	Repeatable happy and exception paths.
6. Specialist logic	Inventory/procurement/finance/logistics rules and tests.	Deterministic policy checks pass.
7. Knowledge layer	Neo4j sync and optional pgvector retrieval.	Results trace back to canonical records.
8. Audit/realtime	Append events, WebSocket updates, optional hash anchoring.	Workflow can be reconstructed and evidence verified.
9. Hardening	Negative tests, auth review, secret scan, monitoring, docs.	Critical cases fail safely.
10. Demo/release	Seed scenarios, rehearse runbook, capture test evidence.	Clean end-to-end demonstration with honest status labels.

Priority if time is constrained

1. Make one order genuinely end-to-end with backend persistence.
2. Implement deterministic inventory/finance rules and safe exception handling.
3. Persist and display audit events linked to the workflow.
4. Connect the frontend to the API and label remaining simulation.
5. Add the specialist orchestration and repeatable scenarios.
6. Add graph, vector, WebSocket, and blockchain only after the core flow is reliable.

20. Final Demo Checklist and Documentation Artifacts

Demo checklist

- Can a new developer run the project using the documented steps?
- Can the presenter explain each technology and distinguish live from simulated integrations?
- Can ORD-1042 be traced through specialist checks to a final status?
- Can all four exception scenarios be triggered and explained?
- Does each exception have a cause, severity, owner, and evidence reference?
- Can the presenter show the exact specialist results behind the Supervisor decision?
- Can the audit timeline be filtered by order and workflow run?
- Does persisted data survive a backend restart?
- Do negative tests prove that finance rejection and agent failure cannot silently pass?
- Are environment variables, known limitations, architecture, and test commands documented?

Suggested companion docs

- README.md — prerequisites, setup, environment, run commands, tests, troubleshooting, and status matrix.
- docs/architecture.md — component diagram, data flow, boundaries, and failure paths.
- docs/data-model.md — ER diagram, tables, migrations, and data ownership.
- docs/agents.md — responsibilities, input/output contracts, tools, policies, examples.
- docs/workflows.md — state machine and scenario walkthroughs.
- docs/api.md — routes, schemas, auth, errors, and examples.
- docs/security.md — secrets, access controls, prompt injection, audit retention.
- docs/testing.md — commands, fixtures, scenario matrix, known gaps.
- docs/deployment.md — local/staging/production configuration, rollback, and health checks.

Appendix A. Status Legend

Status	Meaning
Implemented and verified	Code exists and repeatable test or live demonstration proves it works.
Implemented, not integration-tested	Code exists but end-to-end behaviour with dependencies is not verified.
Prototype / simulated	UI or workflow uses demo state or simulated outputs.
Starter / scaffold	Only a base module, schema, contract, or sample implementation exists.
Planned	Target capability not verified in the current branch.
Blocked	Waiting on dependency, credentials, schema decision, or unresolved defect.

Use these labels consistently in the README, presentation, and project report. A screen that looks complete is not evidence that its backend integration is complete.